흐름을 읽고,
필요를 만드는 개발자
박재영 입니다

Park Jaeyoung · 국민대학교 소프트웨어학과

leeds102401@gmail.com · 010 2165 2750

ABOUT

프로필



박재영

2001. 10. 24

국민대학교 소프트웨어학과

github.com/co2ma

"흐름을 읽고, 필요를 만드는 개발자"

- 좋은 서비스의 시작은 시장과 수요를 이해하는 것이라고 생각합니다
- 사용자가 필요로 하는 기능과, 그것을 쉽게 쓰게 하는 흐름을 먼저 봅니다
- 기능이 동작하는 것에 그치지 않고, 같은 입력에 같은 결과가 나오고 실패도 정해진 형태로 돌아오는 구조를 만듭니다
- 유용한 소프트웨어를 목표로, 흐름을 읽는 개발자가 되고자 합니다

학력 · 교육

2026.01 — 현재	삼성청년SW·AI아카데미(SSAFY) 15기 (1학기 수료, 2학기 진행 중)
2025.07 — 2025.08	네이버 부스트캠프 챌린지 과정 수료
2020.03 — 2026.02	국민대학교 소프트웨어학과 졸업
2021.01 — 2022.07	육군 미사일전략사령부 병장 전역

자격증

2026.09.11	정보처리기사
2026.06.05	데이터분석준전문가 (ADsP)
2026.03.27	SQL 개발자 (SQLD)

수상

2026.08.10	SSAFY 공통 프로젝트(YORR) 우수상 (1등)
2026.06.26	SSAFY 1학기 성적우수상 (3등)
2025.11.20	학술대회 우수논문상
2025.05.30	다학제간캡스톤디자인 졸업전시회(WIGo) 동상
2019.08.24	제4회 국민대학교 알고리즘 대회 장려상

TECH STACK

기술 스택

기술	숙련도	사용 경험
Java · Spring Boot	■■■■□	RESTful API 설계, WebSocket 실시간 처리, 스케줄러 기반 배치, 서비스 개발·배포
SQL	■■■□□	연관관계·정규화 기반 스키마 설계, Flyway로 DB 변경 이력 관리
Redis	■■■□□	방·세션·게임 상태 저장, TTL 설계와 분산 락 기반 동시성 처리
Docker	■■■□□	DB·Redis를 포함한 개발 환경 구성과 배포 환경 컨테이너화
Git / GitHub	■■■□□	브랜치 전략에 따른 협업, PR 리뷰와 충돌 해결
Jira	■■■□□	스프린트 단위 이슈 관리와 진행 상황 공유
React	■■□□□	Axios 기반 API 연동 화면 구현

알고리즘 1,000+ 문제

알고리즘 문제 1,000문제 이상 풀이. DFS/BFS·그래프·이진탐색 등 폭넓게 다루며 알고리즘 대회 장려상을 수상했습니다.

유지보수 가능한 구조

계층 분리 설계, 마이그레이션 이력 관리, API 문서화를 중요시합니다.

기술보다 시장 먼저

만들기 전에 트렌드를 읽고 사용자부터 정의합니다. 기술은 그 필요를 실현하는 수단입니다.

PROJECTS

진행한 프로젝트

01

YORR

휴대폰을 흔들어 함께 하는 실시간 웹 게임 플랫폼. (SSAFY 프로젝트 1등)

6인 팀 · 백엔드 코어 담당

2026.07 — 2026.08

02

다짐 캡슐

다짐을 기록하고 정해진 날짜에 이메일로 돌려받는 온라인 타임캡슐 서비스

1인 개인 프로젝트 · 기획—배포 전담

2026.04 — 2026.06

03

WIGo

VI-SLAM 기반 시각장애인·노약자용 실내 내비게이션 서비스 (캡스톤 동상)

5인 팀 · App UI / 음성 / LLM 서버 전담

2025.01 — 2025.06

PROJECT 01 · 프로젝트 개요

YORR

스마트폰 센서로 조작하는 실시간 멀티 인터렉티브 게임 서비스

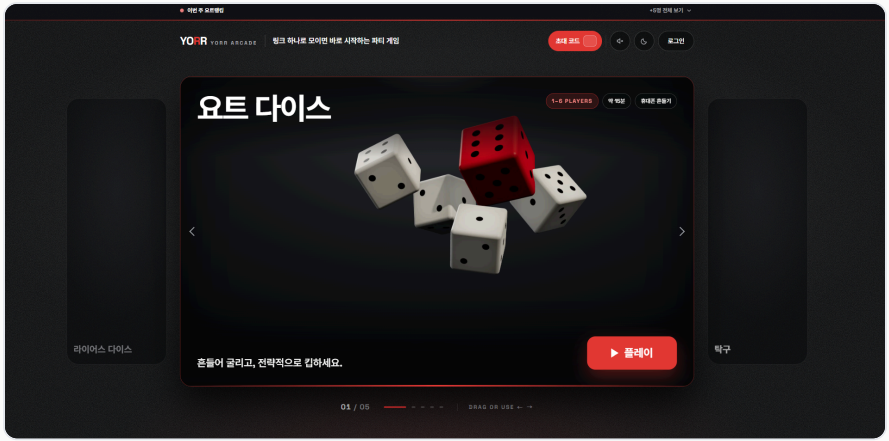
26.07 — 26.08 · 6인 팀

담당 — 백엔드 코어 (방·세션·게임 상태 / 게임 모듈화 / 빠른 대전 매칭)

직접 구현한 기능

- 방 생성·참가, 초대 코드 발급·검증과 게스트 세션 (Redis)
- WebSocket 기반 실시간 게임 상태 판정과 재접속 복구
- 게임 공통 인터페이스 설계와 게임별 모듈 분리
- Redis 대기열·분산 락 기반 빠른 대전 매칭

Java · Spring Boot · WebSocket · Redis · MySQL · JUnit 5 · TypeScript



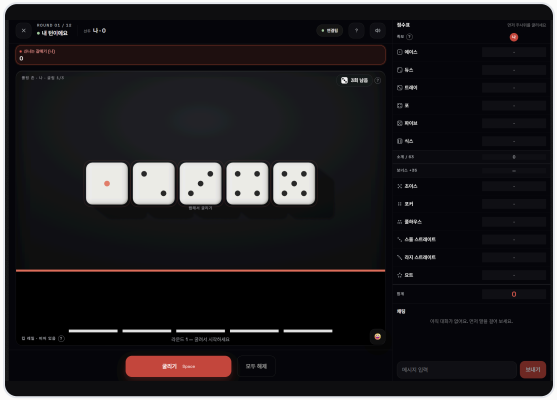
게임 메인 화면 (yorr.site)

서비스 성과

80명+ 최고 동시접속

487판+ 누적 플레이

635건+ 게임 참여



요트 다이스

1~6명 · 화면 탭 · 휴대폰 흔들기



탁구

1~2명 · 화면 탭 · 휴대폰 스윙



석양이 진다 (순발력 게임)

2명 · 화면 탭 · 휴대폰 휘두르기

2주간 2번의 배포를 통한 유저 테스트

7월 30일 요트 다이스로 1차 배포, 8월 5일 2차 배포에 탁구와 석양이 진다를 추가 (현재도 계속 운영중)

시스템 구조

휴대폰 조작 (흔들기 · 탭) → 서버가 누구의 입력인지 확인 → 게임 규칙으로 점수 판정 → Redis에 결과 저장 → 같은 방 전원에게 전송

방 · 게스트 세션

방을 만들면 초대 코드가 나오고, 회원가입 없이 코드만으로 참여

실시간 게임 상태

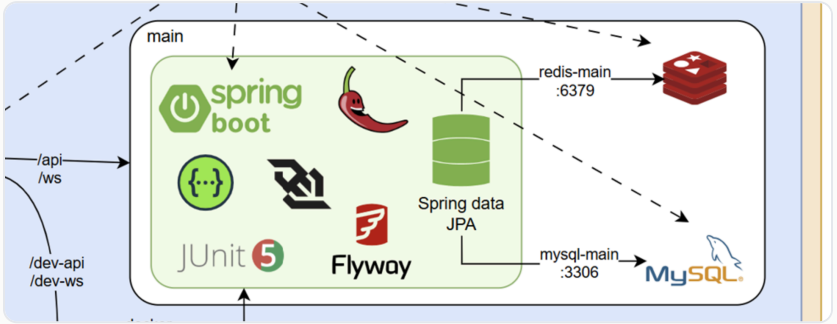
휴대폰 조작을 서버가 판정해 점수와 순서를 결정하고, 결과를 Redis에 저장해 재접속 시 복구

게임 모듈화

공통 부분(방·순서·점수)과 게임별 규칙을 분리해, 새 게임은 규칙만 만들어 등록하면 연결

빠른 대전 매칭

혼자 들어와도 대기열에서 상대를 찾아 같은 방으로 연결, 한 사람이 두 방에 들어가지 않도록 락으로 차단



아키텍처(일부) 담당 구간 — Spring Boot · Spring Data JPA · Redis · MySQL

트러블 슈팅

WebSocket 재연결 시 방 인원 중복 집계

문제 재연결된 사용자를 새 사용자로 인식해 방 인원이 늘고 점수가 초기화됨

해결 **기존 게스트 ID 재사용** + 방 상태 재전달로 끊긴 지점부터 이어가도록 수정

이전 재연결 → 새 ID 발급 → 3명 · 점수 초기화

이후 **재연결 → 기존 ID 재사용 → 2명 · 상태 복구**

게임 추가 시 조건 분기 증가

문제 요트 규칙이 WebSocket 핸들러에 섞여 있어 게임을 추가할 때마다 분기가 늘어남

해결 **공통 인터페이스와 레지스트리**로 게임별 모듈 분리 → 2차 배포 2종을 구조 변경 없이 추가

빠른 대전 종료 후 대기 상태 잔존

문제 이미 입장한 사용자가 다음 매칭에 또 선택되어 두 방에 동시 소속됨

해결 방 정리와 대기 상태 삭제를 **한 곳에서 함께 처리**하도록 수정 후 통합 테스트로 검증

동시 입장 시 방 최대 정원 초과

문제 동시 입장 시 인원 조회와 증가 사이에 다른 요청이 끼어들어 정원을 넘겨 입장됨

해결 정원 확인과 입장을 **Redis Lua script로 한 번에 처리**해, 초과 시 스크립트에서 바로 거절

회고

상태 저장 위치

게임 정보를 서버 메모리가 아닌 Redis에 저장해, 접속이 끊기거나 서버를 다시 켜도 게임이 사라지지 않도록 구성

게임 추가를 위한 구조 분리

게임마다 다른 규칙만 따로 떼어내, 새 게임을 추가할 때 방·순서·점수 같은 공통 코드를 그대로 재사용

동시 요청 처리 방식

점수 변경은 순서를 기다리게, 매칭은 기다리지 않고 건너뛰게 처리하며 작업 성격에 따라 방식이 달라진다는 점을 체감

남은 과제 — 사람이 챙겨야 하는 정리

게임 정보를 지우는 코드를 매번 직접 불러야 하는 구조. 한 번 빠뜨리면 데이터가 남아, 저장 시 함께 처리되도록 개선할 필요성을 확인

돌아보며

가장 어려웠던 건 게임 규칙이 아니라 데이터를 어디에 두고, 언제, 누가 지울지 정하는 일이었습니다. 이 세 가지를 미리 정해둔 곳에서는 버그가 거의 없었고, 대충 넘긴 곳에서는 어김없이 문제가 생겼습니다.

백엔드를 담당했지만 후반에는 인게임 오디오(BGM, 효과음)까지 맡아, 사용자가 실제로 체감하는 부분까지 완성했습니다. 앞으로는 데이터를 언제, 누가 지울지 설계 단계에서 먼저 정하고, 그 규칙이 코드로 지켜지게 만드는 개발자가 되고자 합니다.

다짐 캡슐

개인·그룹의 다짐을 기록하고, 정해진 날짜에 이메일로 돌려주는 온라인 타임캡슐

숏폼 콘텐츠에 지친 세대에게 '의도된 기다림'과 '자기 성찰'의 경험을 돌려주고 싶어 기획했습니다.

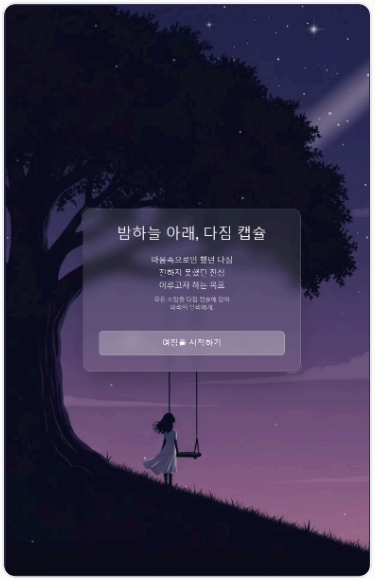
26.04 — 26.06 · 1인 개인 프로젝트

담당 — 기획 · 개발 · 배포 전 과정

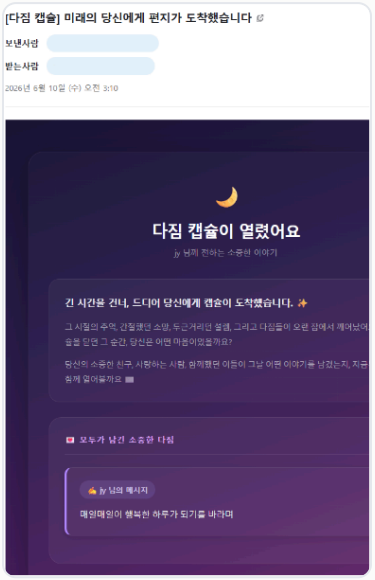
직접 구현한 기능

- 다짐 작성·조회 REST API와 계층 분리 설계 (Controller—Service—Repository)
- 지정일 이메일 발송 배치 — cron 트리거 / 대상 분류 / 발송 실행 3계층 분리
- 재전송 가능한 DB 상태 설계와 Flyway 기반 스키마 이력 관리
- React·Axios로 작성·조회 화면 구현, Swagger로 API 문서화

Java Spring Boot JPA PostgreSQL Flyway React Docker



메인 화면



결과 화면 (이메일 전송)

시스템 구조

전체 기본 구조

RESTful API 설계, Controller → Service → Repository 계층을 기능별로 명확히 분리

이메일 배치 — 단일 책임 분리

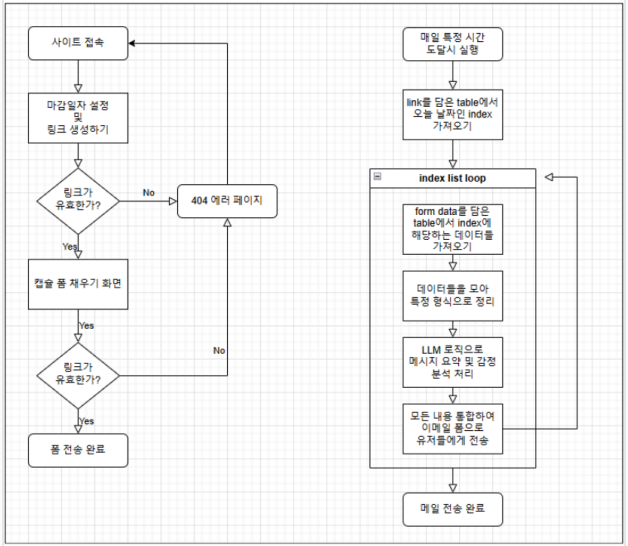
cron 트리거 / 발송 대상 분류 / 발송 실행의 3계층으로 분리, 30분 주기로 실행해 누락 없이 전송

재전송 가능한 DB 설계

is_done(캡슐)·is_processed(메시지) 이중 상태 컬럼으로, 서버 장애 시에도 실패분만 재전송 가능한 구조

설정 분리

DB·메일 계정 정보를 .env로 분리해 코드에 노출되지 않도록 구성 (spring-dotenv)



사용자 플로우 · 이메일 배치 흐름도

트러블 슈팅

배포 환경 스케줄러 미동작

문제 날짜가 바뀌어도 cron이 의도대로 동작하지 않음

해결 @Scheduled에 **타임존 명시** + docker-compose에서 **KST 강제 설정**

이전 KST 00:00 예약 → 서버 UTC 판단 → 9시간 뒤 발송

이후 **KST 00:00 예약 → KST 고정 → 정시 발송**

Hibernate 테이블명 자동 변환 충돌

문제 엔티티가 실제 테이블(capsule_link)이 아닌 unique_link를 탐색해 앱 기동 실패

원인 Hibernate가 클래스명을 snake case로 자동 변환

해결 **@Table로 명시** 후, **Flyway 마이그레이션**으로 클래스—테이블명 정합화

Builder 패턴 오사용으로 값 유실

문제 new로 생성한 객체에 .builder()를 호출해 빈 객체가 저장됨

원인 builder는 기존 객체 수정이 아니라 불변 객체를 새로 생성하는 패턴

해결 CapsuleLink.builder()…build() 반환값을 받는 방식으로 수정

발송 실패 메시지 영구 미처리

문제 전송 실패 시 예외만 catch하고 넘어가 해당 편지가 영구 미처리 상태로 남음

원인 예외 발생 시 상태 갱신 없이 흐름이 종료

해결 **실패 상태를 별도 마킹**하고 캡슐 ID를 로깅해 재처리 가능한 예외 흐름으로 수정

회고

문서화의 가치

프론트·백엔드를 오가며 endpoint를 헷갈릴 때마다 Swagger 문서가 이정표가 됐고, 협업에서의 문서화 가치를 체감

정규화와 마이그레이션

제2정규화·이행적 종속 분리를 직접 수행하고, Flyway 도입으로 DB 변경을 이력으로 관리

단위 테스트 도입

수동 요청 확인에서 놓치던 실수를 기능별 단위 테스트로 방지

남은 과제 — 인증·생성 제한

비로그인 설계상 링크만 있으면 무한 생성이 가능한 구조. IP 제한 등 안전장치의 필요성을 확인

돌아보며

기능 동작에 집중하다 보니 인증·예외 처리처럼 서비스를 안전하게 만드는 요소가 후순위로 밀렸습니다. 프론트와 백엔드를 함께 개발하며 상태 코드 하나, 응답 구조 하나가 클라이언트 전체에 영향을 준다는 것을 체감했고, **좋은 개발이란 기능이 동작하는 것이 아니라 사용하는 쪽이 편한 구조**임을 이해하게 되었습니다.

앞으로는 기능을 만드는 동시에 예외 흐름, 일관된 응답 구조, 인증 설계까지 함께 챙기는 습관을 들이려 합니다.

WIGo Where I Go

VI-SLAM 기반 시각장애인·노약자용 실내 내비게이션

GPS가 닿지 않는 실내에서, 자율주행에 쓰이는 SLAM 기술과 스마트폰 IMU 센서를 결합해 위치를 추정하고 음성만으로 길을 안내합니다. 캡스톤디자인 졸업전시회 동상 수상 및 한국통신학회 논문 수상작입니다.

25.01 — 25.06 · 5인 팀

담당 — 사용자 접점 전담 (App UI 설계·구현 / STT·TTS 음성 인터페이스 / LLM 서버 구축)

직접 구현한 기능

- Android 앱 화면 설계·구현 (큰 버튼·고대비 중심의 접근성 UI)
- STT 인식 → LLM 목적지 추출 → TTS 안내 파이프라인 구성
- FastAPI + GPT-4o mini 기반 LLM 서버 단독 구축·배포
- TTS 발화 큐 처리와 안내 타이밍 설계

Kotlin Android Python FastAPI GPT-4o mini AWS S3



제가 구현한 앱 화면 — 음성 명령 처리와 실내 안내 UI

담당 구현

음성 인식 (STT) → LLM 목적지 추출 (JSON) → Pose Graph 위치 추정 → A* 경로 탐색 → 음성 안내 (TTS)

LLM 목적지 추출 서버 (담당)

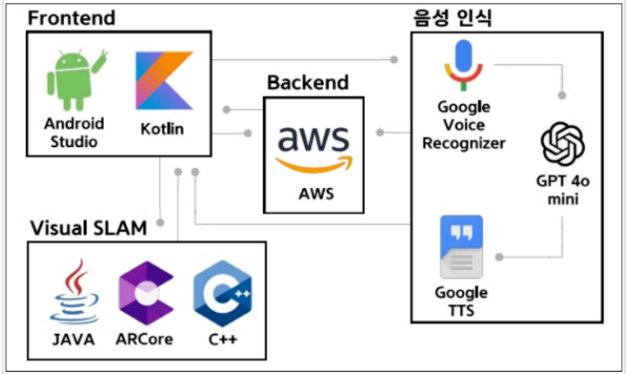
FastAPI와 GPT-4o mini로 서버를 단독 구축해, 자연어 발화에서 목적지를 구조화된 JSON으로 추출하도록 설계

음성 인터페이스 파이프라인 (담당)

STT 인식 결과를 LLM 서버로 보내 목적지·의도를 구조화된 JSON으로 받고, TTS로 안내를 출력하는 파이프라인 구성

접근성 중심 앱 UI (담당)

큰 버튼·고대비 색상 중심으로 설계하고, 음성 명령만으로 모든 기능을 조작할 수 있는 화면 구조 구성



아키텍처 담당 구간 - App · 음성 인식 · LLM 서버

트러블 슈팅

앱에 포함된 API 키 노출

문제 앱이 외부 LLM API를 직접 호출하는 구조라 API 키가 APK에 포함됨 — 추출 시 과금 도용 가능

해결 키를 즉시 폐기하고 **중계 서버를 별도 구축**, 키는 서버에만 두고 앱은 자체 API만 호출하도록 변경

이전 앱 + API 키 → 외부 API → 키 노출

이후 **앱 → 중계 서버(키 보관) → 외부 API**

자연어 발화에서 목적지 파싱 실패

문제 “OOO 4층 가고 싶어” 같은 자연어에서 목적지를 찾지 못해 길찾기 실패

원인 단순 문자열 비교로는 다양한 발화 패턴을 매칭 불가

해결 LLM 프롬프트로 목적지 키워드만 추출, **구조화된 JSON 응답**으로 안정적 파싱

TTS 안내 음성 끊김

문제 다음 안내가 시작되면 재생 중인 앞 문장이 중단됨

원인 speak() 직접 호출이 이전 음성을 덮어씀

해결 발화를 **Queue에 쌓아** 이전 발화 종료 후 이어지도록 처리

S3 Pose Graph 다운로드 차단 (CORS)

문제 앱에서 층별 Pose Graph 파일 요청이 차단됨

원인 S3 버킷에 CORS 정책 미설정 — 클라이언트 직접 요청 거부

해결 **CORS 정책 추가**, 민감 버킷은 **EC2 경유** 제공으로 변경

회고

타겟 사용자 관점

화면을 보지 않고 조작해야 하는 사용자에게는 음성 타이밍 하나가 사용성을 좌우함. 기능이 동작하는 것과 타겟 사용자가 실제로 쓸 수 있는 것은 다르다는 것을 배움

낯선 기술의 단독 학습

FastAPI와 OpenAI API를 처음 다루며 문서 기반으로 혼자 LLM 서버를 구축. 낯선 기술이라도 역할이 주어지면 해낼 수 있다는 자신감 확보

인터페이스 합의의 중요성

파트 간 입출력 형식이 어긋나면 전체 흐름이 끊김. 개발 전에 형식을 합의하는 습관의 중요성을 체득

남은 과제 — 실사용자 테스트

팀원끼리만 테스트해 실사용자 피드백 기반의 접근성 보완이 부족했던 점이 아쉬움

돌아보며

팀의 기술적 무게중심이 SLAM에 있었던 만큼, 저는 그 결과물을 사용자에게 전달하는 접점을 만들었습니다. 처음 접하는 Android와 LLM 서버 구축을 병행하며 낯선 환경에서 스스로 학습해 기여하는 방식을 익혔고, **기술을 빠르게 습득하는 것보다 그것을 사용자 흐름에 자연스럽게 녹여내는 것이 중요하다**는 것을 배웠습니다.

앞으로는 사용자와 시스템을 잇는 백엔드 구조를 더 깊이 고민하는 개발자로 성장하고 싶습니다.

감사합니다

박재영

leeds102401@gmail.com · 010 2165 2750
github.com/co2ma